Engineering Approach & System Design Document

Document Versioning, Ingestion, and Verification Engine

Prepared by: Balaraj M P, BE Information Science and Engineering, CMR Institute of Technology
Status: Final Submission
Target Platform: FastAPI + SQLite / MongoDB & Gemini LLM Pipeline
Date: July 2026

Table of Contents

System Overview

Tri9T is a professional automated verification platform designed to ingest complex technical PDF manuals, reconstruct their hierarchical outline structure, and manage logical node revisions over time. It enables QA test engineers to highlight specific sections of a manual, generate 3-5 high-fidelity test cases using Gemini LLM structured outputs, and automatically trace the staleness of these test cases as updated versions of the source manuals are uploaded.

1. Executive Summary & System Architecture

The platform employs a clean, decoupled service architecture to separate core ingestion, validation, and retrieval tasks:

- **API Router Layer:** Implements FastAPI routers for selections, document versions, node retrieval, and QA management. It strictly enforces Pydantic request and response models.
- **Service Layer:** Orchestrates parsing logic (via PyMuPDF and PyTesseract), tree reconstruction (via Stack-based hierarchy parsing), version comparisons, and LLM retry pipelines.
- **Persistence Layer:** Operates an asynchronous SQLite engine via aiosqlite for version control mappings, selections, and test case records. In parallel, MongoDB stores raw extracted page layouts and blocks.

2. Relational Database Schema

The database tracks the evolution of document outlines across multiple manual uploads. Rather than tracking changes line-by-line, the schema maps revisions to **Logical Nodes**. A logical node acts as a persistent anchor with a stable UUID. When a new document version is imported, the system maps matching outline sections back to the same logical node ID, creating a new NodeVersion record that links to the new DocumentVersion.

Table	Key Columns	Description
documents	id (PK), name	Represents the root document profile.
document_versions	id (PK), document_id (FK), version_number, commit_message	Captures a specific manual revision or upload transaction.
logical_nodes	id (PK), uuid, document_id (FK)	Stable anchor representing a structural node across all document versions.
node_versions	id (PK), logical_node_id (FK), document_version_id (FK), parent_logical_node_id (FK), title, content, content_hash	Holds the actual text content, title, and SHA-256 hash of a node under a specific version.
selections	id (PK), document_version_id (FK), name	A user-selected subset of logical nodes pinned to a document version.
generated_test_cases	id (PK), selection_id (FK), question, answer, reference_context	Stores the validated, LLM-generated QA test cases linked to a selection.
llm_generation_failures	id (PK), selection_id (FK), raw_response, error_message	Logs failed LLM outputs and schema validation errors for audits.

3. PDF Ingestion & Hierarchy Reconstruction

Parsing complex layout specifications (like the CT200 user manual) requires a five-step extraction pipeline. The core classes implemented inside *app/services/pdf_parser.py* are **LayoutAnalyzer**, **TableDetector**, **BlockClassifier**, and **HierarchyBuilder**.

Spatial Bounding Box Filtering for Table Content Duplication

The initial implementation duplicated table contents: cell text was parsed as tabular grids and also extracted independently as standard text paragraphs. After visual verification, the **LayoutAnalyzer** was updated with bounding-box filtering. The system computes the spatial center of every text block. If that center falls within the bounding box of a detected table, the block is dropped from the paragraph stream, ensuring table cells are not duplicated.

4. Parser Edge Cases & Mitigations

Edge Case Scenario	Pipeline Mitigation Strategy
Missing intermediate headings E.g. heading 3.1.2 appears but 3.1 was skipped.	The HierarchyBuilder detects the missing parent key, raises a hierarchy mismatch warning, and attaches the node to the longest matched prefix (e.g. heading 3.0 or root) to preserve order.
Page-boundary text split A paragraph breaks across page bounds.	Parser checks if the next block starts with a lowercase letter, a hyphen, or a list marker, automatically merging it into the previous paragraph block.
Duplicate headings Multiple headings share a key or name.	The system maintains local counters for each heading path and appends a duplicate suffix (*_dup[count]*) to enforce unique, stable logical signatures in the database.
Table text duplicate extraction Standard readers extract cell text twice.	The LayoutAnalyzer performs a spatial boundary check. If a text block's coordinate center falls within the bounding box of a detected table, it is dropped from the text stream.
Ordered list headings mismatch Numbered list items match heading patterns.	List items starting with numbering are filtered out from being headings if they exceed 80 characters, contain colons, or contain internal newlines.

5. Validation Methodology

To verify that PDF parsing, table extraction, hierarchy builders, and version comparisons work correctly, the system incorporates the following checks:

- **Manual Outline Comparison:** Directly compared output JSON outline structures against the visual hierarchy of the source PDF files.
- **Visual Layout Analysis:** Rendered page blocks to verify correct column reading order and header/footer exclusion.
- **Hierarchy Verification:** Validated that sub-sections (e.g., 2.1.1.1) are correctly structured as children of 2.1.1, throwing warnings on mismatches.
- **Table and List Verification:** Verified cell alignment in pipe format and confirmed ordered list items do not pollute headings.
- **Version Match Audits:** Tested version diffing endpoints to verify they capture unchanged, modified, added, and removed nodes correctly.
- **Pytest Suite:** Isolated unit tests assert every pipeline class (LayoutAnalyzer, TableDetector, BlockClassifier, HierarchyBuilder) against synthetic inputs.

6. Document Versioning & Matching Strategy

A key design requirement is recognizing logically identical nodes when a new document version is ingested. Rather than performing character-based text alignment, the system maps outline nodes based on structural path signatures.

1. **Path Signature Construction:** During manual ingestion, a signature string is generated for each node representing its type, heading hierarchy, and order index relative to its parent (e.g., *heading:2.0 > heading:2.1 > paragraph:3*).
2. **Map Alignment:** The signature is looked up in the previous version's signature-to-logical-node map (*prev_sig_map*).
3. **Identity Mapping:** If found, the node is associated with the existing *logical_node_id*. This preserves history across re-uploads. If missing, a new *logical_node_id* is created.

Change Detection Rule Matrix

Scenario	Signature Path Match	Content Hash Match	Result Category
Identity Preserved, No Change	Matches Previous	Identical	**Unchanged**
Node Relocated in Manual	Different Path	Identical	**Unchanged (Moved)**
Section Modified in Place	Matches Previous	Differs	**Modified**
New Section Introduced	No Match (New)	N/A	**Added**
Section Removed / Deleted	No Match in V2	N/A	**Removed**

7. Selection Management

Users select logical nodes to pin baseline requirements. The primary routes are **POST /api/v1/selection** and **GET /api/v1/selection/{id}**. Selections remain **immutable** because they reference a specific *document_version_id* and the text content of those nodes at that time, protecting baselines from source manual modifications.

8. Structured LLM Generation & Pydantic Validation

The Gemini model is prompted to act as a Senior QA Automation Engineer, receiving context and returning structured JSON matching a Pydantic schema:

```
{
  "properties": {
    "test_cases": {
      "items": {
        "properties": {
          "question": { "type": "string" },
          "answer": { "type": "string" },
          "reference_context": { "type": "string" }
        },
        "required": ["question", "answer", "reference_context"],
        "type": "object"
      },
      "type": "array"
    }
  },
  "required": ["test_cases"]
}
```

The system strips markdown formatting and validates the JSON structure. If validation fails, it executes **exactly one retry**. If that also fails, the details are logged to *llm_generation_failures*, and an HTTP 422 error is returned.

9. Stale Traceability Detection

Upon manual revision ingestion, selection status is computed:

- **Fresh:** All selected logical nodes exist in the target version, and their content hashes (SHA-256) match.
- **Possibly stale:** All selected nodes exist, but at least one node has a different content hash.
- **Stale:** One or more selected nodes are completely missing (deleted) in the new version.

Limitations of Hash Detection: Hashing triggers false positives on minor cosmetic changes (spacing/punctuation), misses semantic re-ordering context (false negatives), and is blind to sibling or parent heading updates.

10. API Endpoint Documentation

GET /api/v1/documents

• *Purpose:* Lists all ingested SQL documents.

• *Response:* [{"id":1, "name":"ct200_manual.pdf"}]

GET /api/v1/versions

• *Purpose:* List all document versions. Supports document_id filter. (Note: GET /documents/{version} is not implemented directly; instead versions are browsed using GET /api/v1/versions?document_id={id}).

• *Response:* [{"id":1, "version_number":1, "commit_message":"Import"}]

GET /api/v1/nodes/{id}

• *Purpose:* Fetch details and historical node versions of a logical node by UUID.

• *Response:* {"id":5, "uuid":"abc-123", "node_versions":[{"id":12, "title":"Battery Life"}]}

GET /api/v1/search?q={query}

• *Purpose:* Performs search across outline nodes' contents.

POST /api/v1/selection & GET /api/v1/selection/{id}

• *Purpose:* Create and retrieve selection sets. Selection route prefixes are singular in the implementation.

• *Request Payload:* {"name":"Specs", "document_version_id":1, "nodes":[{"node_id":"abc-123"}]}

GET /api/v1/generation/{selection_id} & GET /api/v1/generation/node/{node_id}

• *Purpose:* Retrieve generated test cases, versioning details, staleness status, and diff summaries. Prefix routes are singular in the implementation.

• *Response:* {"selection_id":1, "staleness_status":"Possibly stale", "test_cases":[...]}

11. Unit Tests & Verification

The test suite verified: • **Deep Hierarchy:** Section 2.1.1.1 structures as a fourth-level child node (*test_heading_2_1_1_1_becomes_fourth_level_node*). • **Duplicate Headings:** Identical titles (Overview) produce unique stable keys and logical node UUIDs (*test_two_headings_with_identical_titles_produce_different_node_ids*). • **Reading Order:** Sections occurring out-of-order (3.4 before 3.3) preserve input reading sequence (*test_heading_3_4_appearing_before_3_3_preserves_reading_order*). • **Tables & Lists:** Verified Markdown pipe output and ensured ordered lists are not classified as headings.

12. Technical Decision Log

Q1: What is the one part of this system most likely to silently give wrong results without erroring? How would you detect it?

Answer: The layout sorting and block classification. If bounding boxes are slightly off, the reader might sort sentences out of order or classify a heading as list text. It executes fine, but the hierarchy is corrupt. *Detection:* Render PDFs with debug overlay boxes (color-coded by classified block type), run structural sequence alerts (e.g. 2.1.2 followed by 2.1.9), and monitor text lengths statistically.

Q2: Where did you choose simplicity over correctness because of time? What would break first if this went to production?

Answer: Stable logical node signature matching. We use path signatures like *heading:2.1 > leaf:2.1_paragraph_1*. *Production Failure:* If a new paragraph is inserted at the start of a section, all downstream siblings shift indexes, losing their logical identity. They are flagged as 'Removed'/'Added', causing their test cases to report 'Stale' falsely.

Q3: Name one input (parser, version matcher, or LLM call) that your implementation does NOT handle. What happens when it encounters it?

Answer: Multipage spanning tables and nested sub-tables. *Encounter behavior:* The TableDetector treats the table segments on subsequent pages as independent tables under their respective parent headings. Nested tables output malformed pipe markdown, cluttering structural text.

13. Known Limitations & Future Improvements

• **Known Limitations:** Cosmetic changes trigger false-positive staleness reports; out-of-selection context modifications (parent updates) are missed.
• **Future Improvements:** Calculate cosine similarity of embeddings to filter out cosmetic updates; compute relative outline hashes to detect hierarchy shifts; run LLM workers to auto-repair stale QA cases.